claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-a77adcbc098c7cca7.jsonl`
- SHA-256: `2bc65fad45c019dd7b4bf421dadc1c3efca0416398ebe0936f9bdb2cd704bc84`
- Source modified: `2026-07-08T06:15:16+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_5f13fc79-bad`
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`

Transcript

1. user (2026-07-08T06:12:31.328Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to
an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).
The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):
- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical
  to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the
  SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config
  + source uri) and needs NO control-plane DB. It reuses orchestration._stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed
  once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing,
  never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a
wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT
report style, naming, or speculative concerns. Cite exact file:line from undefined.

A reviewer claims this DEFECT in the merged #521 code. Try HARD to REFUTE it by reading the actual code at undefined. Default to is_real=false if you cannot concretely confirm the failure. Skeptic #2.

CLAIM (low/robustness) backend/worker/__main__.py:488

summary: cmd_compile writes its done-marker ONLY after _stitch_reel returns; any failure BEFORE that (spec fetch/parse, time_pairs/stitching coercion, VideoCompiler init) raises out of an unwrapped main(), so the worker exits with a traceback and NO marker ? unlike cmd_infer's _fail() which always writes a marker for every worker-side failure.
failure scenario: The bucket-staged spec is truncated/corrupt (transient partial read) or time_pairs contains a malformed pair. `_read_compile_spec` (worker:426-432) raises JSONDecodeError/ValueError, or the coercion `[(float(pair[0]), float(pair[1])) for pair ...]` (worker:493-495) / `StitchingConfig(**...)` (worker:496) raises IndexError/TypeError. cmd_compile has no surrounding try/except and main() at line 973 calls args.func(args) unwrapped, so the process exits non-zero with a raw traceback and never writes the --done-uri marker. The dispatcher's bucket-poll then depends ENTIRELY on the #515 crash-fast Executions-API probe to convert this into a clean failed job; if that probe is unavailable/delayed, the poll hangs up to deployment.remote_poll_max_wait_sec (~65 min). cmd_infer avoids this asymmetry ? its _fail() writes a real-rc marker for every pre-segmentation failure so the dispatcher terminates fast regardless of the Executions API.

2. assistant (2026-07-08T06:12:39.301Z)

I'll investigate the claim by reading the actual code. The paths show "undefined" (a templating failure), so I'll work with the actual repo. Let me start by examining the key files.

3. user (2026-07-08T06:12:41.710Z)

```
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
---STATUS---
M  README.md
M  SESSION_HANDOFF.md
M  backend/config/models.py
M  backend/infrastructure/database_jobs.py
M  backend/main.py
UU backend/orchestration.py
UU backend/pipeline/stitcher/compiler.py
M  config.json
M  tests/test_api_endpoints.py
M  tests/test_async_jobs.py
M  tests/test_compiler.py
M  tests/test_watermark.py
?? score_golden_gate.py
?? score_local.py
```

4. user (2026-07-08T06:12:41.854Z)

```
backend\worker\__main__.py
```

5. assistant (2026-07-08T06:12:49.433Z)

Let me read the worker main file, focusing on the compile and infer commands and main().

6. user (2026-07-08T06:12:50.423Z)

```
1       """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3           python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4               [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5           python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)
6
7       The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
```

```
``local://``
8       is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9       are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10      ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11      the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13      This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14      (validator registry ? segmenter ? report), never a forked pipeline.
15      """
16
17      from __future__ import annotations
18
19      import argparse
20      import csv
21      import json
22      import logging
23      import os
24      import re
25      import sys
26      import tempfile
27      from typing import Any, List
28
29      from backend import storage
30      from backend.compute import ComputeJobSpec, get_compute_backend
31      from backend.config import (
32          StartupCheckError,
33          enforce_startup_checks,
34          load_config,
35          probe_cuda_available,
36      )
37      from backend.infrastructure.database import Database
38      from backend.pipeline.segmenters.base import segmenter_registry
39      from backend.pipeline.validators.base import registry as validator_registry
40      from backend.reporting import emit_indexer_report
41      from backend.results import Failure
42      from backend.utils.ffmpeg import ffprobe_bin  # P2a: single ffmpeg/ffprobe resolver
43      from backend.utils.hashing import compute_video_id
44
45      logger = logging.getLogger("backend.worker")
46
47
48      def _coerce(v: str) -> Any:
49          """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
50          low = v.lower()
51          if low in ("true", "false"):
52              return low == "true"
53          for cast in (int, float):
54              try:
55                  return cast(v)
56              except ValueError:
57                  pass
58          return v
59
60
61      def _apply_overrides(config: Any, sets: List[str]) -> None:
62          """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
63          for kv in sets or []:
64              key, sep, val = kv.partition("=")
65              if not sep:
66                  raise ValueError(f"--set expects k=v, got {kv!r}")
```

```
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow(
                    [
                        f"{float(s.get('start_time', 0.0)):.3f}",
                        f"{float(s.get('end_time', 0.0)):.3f}",
                        s.get("shot_count", ""),
                        s.get("ending_reason", s.get("shot_count_confidence", "")),
                    ]
                )


    def _write_report_json(
        video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
    ) -> None:
        with open(path, "w") as f:
            json.dump(
                {
                    "video_id": video_id,
                    "status": status,
                    "segment_count": len(segments),
                    # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
                    # control plane re-hydrating from outputs/<md5>/ restores the video row
                    # with its truthful path ? without this, only the segments recover.
                    "video_uri": video_uri,
                    "segments": [
                        {
                            "start": float(s.get("start_time", 0.0)),
                            "end": float(s.get("end_time", 0.0)),
                        }
                        for s in segments
                    ],
                },
                f,
                indent=2,
            )


    def _serialize_and_push_outputs(
        scratch: str,
        out_uri: str,
        video_id: str,
        segments: List[dict],
        status: str,
        storage_cfg: dict,
        video_uri: str = "",
    ) -> List[str]:
        """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
        ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.

        Shared by the SUCCESS path and the failure path (``_fail``) so a detector
```

```
timeout/abort leaves a
130         ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131         (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
132         report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
133         EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
134         out_local = os.path.join(scratch, "outputs", video_id)
135         os.makedirs(out_local, exist_ok=True)
136         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137         _write_report_json(
138             video_id,
139             segments,
140             status,
141             os.path.join(out_local, "report.json"),
142             video_uri=video_uri,
143         )
144         out_prefix = out_uri.rstrip("/")
145         pushed: List[str] = []
146         for fn in sorted(os.listdir(out_local)):
147             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149             pushed.append(uri)
150         return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output."""
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         )  # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
171             return False  # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
180         from the local 2/3), never a raw traceback:
181           * rc 4 ? the bucket poll exhausted its budget (``PolicyTimeout``): the execution
```

```
may still be
182             running (the shim best-effort cancels it).
183           * rc 5 ? the execution has TERMINATED without a marker
(``RemoteExecutionFailed``): a worker
184             crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
185             ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
186         from backend.compute import RemoteExecutionFailed
187         from backend.infrastructure.execution_policy import PolicyTimeout
188
189         spec = ComputeJobSpec(
190             video_uri=args.video_uri,
191             out_uri=args.out_uri,
192             segmenter=args.segmenter,
193             config_overrides=tuple(args.set or ()),
194             skip_validation=bool(getattr(args, "skip_validation", False)),
195         )
196         try:
197             result = compute.run_infer(spec)
198         except RemoteExecutionFailed as e:
199             logger.error(
200                 "[worker] remote execution terminated without a completion marker: %s", e
201             )
202             return 5
203         except PolicyTimeout as e:
204             logger.warning(
205                 "[worker] remote compute did not complete (no marker within budget): %s", e
206             )
207             return 4
208         logger.info(
209             "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
210             result.video_id,
211             result.returncode,
212             result.outputs_uri,
213         )
214         return result.returncode
215
216
217     def _write_done_marker(
218         done_uri: str,
219         video_id: str,
220         returncode: int,
221         segment_count: int,
222         status: str,
223         storage_cfg: dict,
224         scratch: str,
225     ) -> None:
226         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
227         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
228         try:
229             marker = {
230                 "video_id": video_id,
231                 "returncode": returncode,
232                 "segment_count": segment_count,
233                 "status": status,
234             }
235             local = os.path.join(scratch, "_done.json")
236             with open(local, "w", encoding="utf-8") as f:
237                 json.dump(marker, f)
238             storage.put(local, done_uri, config=storage_cfg)
239             logger.info("[worker] wrote completion marker -> %s", done_uri)
240         except Exception as e:  # never fail a good run because the marker push hiccuped
241             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
```

```
242
243
244     def cmd_infer(args: argparse.Namespace) -> int:
245         config = load_config(args.config) if args.config else load_config()
246         _apply_overrides(config, args.set)
247         if not _run_startup_checks(config):
248             return 2
249
250         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
251         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
252         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
253         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
254         compute = get_compute_backend(config)
255         if compute is not None:
256             return _dispatch_remote(compute, args)
257
258         storage_cfg = config.storage.model_dump()
259
260         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
261         os.makedirs(scratch, exist_ok=True)
262         config.output.output_dir = scratch
263
264         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
265         local_video = storage.fetch(
266             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
267         )
268         print(f"[worker] pulled {args.video_uri} -> {local_video}")
269
270         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
271         video_id = compute_video_id(local_video)
272
273         # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
274         # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
275         # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
276         # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
277         # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278         # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
279         # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
280         # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
281         # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
282         if getattr(args, "skip_validation", False):
283             print(
284                 "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
285                 "gate and already validated this input."
286             )
287             val_results, val_context = [], {}
288         else:
289             val_results, val_context = validator_registry.run_all_with_context(
290                 local_video, config
291             )
```

```
292         failures = [r for r in val_results if not r.passed]
293         db = Database(os.path.join(scratch, config.output.db_name))
294         db.add_video(
295             video_id,
296             local_video,
297             "failed" if failures else "processed",
298             [r.model_dump() for r in val_results],
299         )
300
301         def _fail(rc: int) -> int:
302             # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
303             # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
304             # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
305             # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
306             # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
307             # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
308             # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
309             try:
310                 _serialize_and_push_outputs(
311                     scratch,
312                     args.out_uri,
313                     video_id,
314                     [],
315                     "failed",
316                     storage_cfg,
317                     str(getattr(args, "video_uri", "") or ""),
318                 )
319             except Exception as e:  # never mask the real failure on an artifact-push hiccup
320                 logger.warning("[worker] could not push failure artifacts: %s", e)
321             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
322             if getattr(args, "done_uri", None):
323                 _write_done_marker(
324                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
325                 )
326             return rc
327
328         segments: List[dict] = []
329         segmenter_actual = None
330         segmentation_ran = False
331         status = "failed"
332         try:
333             if failures:
334                 print(
335                     "[worker] validation FAILED: "
336                     + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
337                 )
338                 return _fail(2)
339             seg_name = args.segmenter or config.indexing.default_segmenter
340             segmenter_cls = segmenter_registry.get(seg_name)
341             if not segmenter_cls:
342                 print(
343                     f"[worker] unknown segmenter: {seg_name!r} "
344                     f"(have {list(segmenter_registry.list_available())})"
345                 )
346                 return _fail(2)
347             segmenter_actual = seg_name
348             result = segmenter_cls(db, config).process_video(
349                 video_id, local_video, max_frames=args.max_frames
```

```
            )
            segmentation_ran = True
            if isinstance(result, Failure):
                print(f"[worker] segmenter FAILED: {result.message}")
                return _fail(3)
            segments = result.value if hasattr(result, "value") else result
            status = "success"
            # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
            # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
            # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
            if not segments:
                detector_impl = (
                    os.environ.get("RALLY_DETECTOR_IMPL")
                    or getattr(config.indexing, "detector_impl", None)
                    or "auto"
                )
                logger.warning(
                    "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
                    "The detector substrate yielded no usable trajectories/windows ? check
the detector "
                    "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
                    "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
                    "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
                    "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
                    "detections, or the input resolution differs from the tuned proxy
resolution. "
                    "Re-run with -v for the native command + frame counts.",
                    video_id,
                    segmenter_actual,
                    detector_impl,
                )
        finally:
            # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
            emit_indexer_report(
                db=db,
                video_id=video_id,
                video_path=local_video,
                config=config,
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=args.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=False,
                segmentation_ran=segmentation_ran,
            )

        # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
        pushed = _serialize_and_push_outputs(
            scratch,
            args.out_uri,
            video_id,
            segments,
            status,
            storage_cfg,
            str(getattr(args, "video_uri", "") or ""),
        )

```

```
        print(
            f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
        )
        for u in pushed:
            print("   ", u)

        # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
        # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
        # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
        # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
        if getattr(args, "done_uri", None):
            _write_done_marker(
                args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
            )
        return 0


    def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
        """Fetch + parse the control-plane-staged compile-spec (#521): the resolved
``time_pairs``, the
        resolved ``stitching`` config, the source ``video_uri``, ``video_id``,
``output_name`` + ``locale``.
        A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real
dst)."""
        local = storage.fetch(
            spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
        )
        with open(local, encoding="utf-8") as f:
            data = json.load(f)
        if not isinstance(data, dict):
            raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
        return data


    def _write_compile_marker(
        done_uri: str,
        result: dict,
        video_id: str,
        returncode: int,
        storage_cfg: dict,
        scratch: str,
    ) -> None:
        """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
        dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "status": result.get("status", "failed"),
                "download_url": result.get("download_url", ""),
                "reel_uri": result.get("reel_uri", ""),
                "message": result.get("message", ""),
            }
            local = os.path.join(scratch, "_compile_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote compile marker -> %s", done_uri)
```

```
        except Exception as e:  # never fail a good stitch because the marker push hiccuped
            logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)


    def cmd_compile(args: argparse.Namespace) -> int:
        """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The
stitch is the
        2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
4K reel). Read
        the control-plane-staged compile-spec (resolved time-pairs + stitching config +
source URI), stitch
        the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so
there is no
        forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
the dispatcher's
        bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
requires."""
        from backend import orchestration
        from backend.config import StitchingConfig

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
        os.makedirs(scratch, exist_ok=True)
        # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to
the bucket.
        config.output.output_dir = scratch
        # The dispatcher forwards storage.output_root via --set; --out-uri carries the same
value as a
        # fallback so reel_object_uri (which reads storage.output_root) always resolves the
reel's bucket
        # destination even if the override were dropped.
        if not (config.storage.output_root or "").strip():
            config.storage.output_root = args.out_uri

        spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
        video_id = str(spec.get("video_id") or "")
        out_name = str(spec.get("output_name") or "")
        source_ref = str(spec.get("video_uri") or "")
        locale = spec.get("locale")
        time_pairs = [
            (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
        ]
        stitching = StitchingConfig(**(spec.get("stitching") or {}))

        print(
            f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
            f"encoder={stitching.encoder} out={out_name}"
        )

        result = orchestration._stitch_reel(
            config,
            stitching,
            video_id,
            source_ref,
            time_pairs,
            out_name,
            locale,
            lambda stage: print(f"[worker] compile: {stage}"),
        )
        status = str(result.get("status") or "failed")
        rc = 0 if status == "success" else 3
```

```
515         if status != "success":
516             print(f"[worker] compile FAILED: {result.get('message')}")
517 
518         # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
passes it.
519         if getattr(args, "done_uri", None):
520             _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
521         return rc
522 
523 
524     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
525     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
526     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
527 
528     #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
529     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
530     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
531     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
532 
533 
534     def _label_clip_name(fname: str) -> str:
535         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.
536 
537         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
538         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
539 
540         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-
stamped)
541         * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same
stem)
542         * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
543         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
544 
545         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
546         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
547         """
548         name = fname.replace(".rallies.csv", "").replace(".csv", "")
549         return _LABEL_DATE_PREFIX.sub("", name)
550 
551 
552     def _probe_video_fps_width(path: str):
553         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
554 
555         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
556         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
557         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
558         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
559         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
560         rather than guesses.
561         """
```

```
        import json
        import subprocess

        try:
            out = subprocess.check_output(
                [
                    ffprobe_bin(),
                    "-v",
                    "error",
                    "-select_streams",
                    "v:0",
                    "-show_entries",
                    "stream=r_frame_rate,width",
                    "-of",
                    "json",
                    path,
                ],
                stderr=subprocess.DEVNULL,
            ).decode()
            st = (json.loads(out).get("streams") or [{}])[0]
            num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
            fps = float(num) / float(den or "1")
            width = float(st.get("width") or 0)
            if fps > 0 and width > 0:
                return fps, width
        except (
            subprocess.SubprocessError,
            ValueError,
            KeyError,
            OSError,
            json.JSONDecodeError,
        ):
            pass
        return None


    def _resolve_train_detector(args: argparse.Namespace, config: Any):
        """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
        detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
        stub=CPU mock). Returns the runner, or prints an error + returns None.

        Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
        detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
        no shuttle detector and is rejected.
        """
        from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

        seg_cls = segmenter_registry.get(args.segmenter)
        if not seg_cls:
            print(
                f"[worker] unknown segmenter: {args.segmenter!r} "
                f"(have {list(segmenter_registry.list_available())})"
            )
            return None
        seg = seg_cls(None, config)
        if not isinstance(seg, TrajectoryHybridSegmenter):
            print(
                f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
                f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
            )
            return None
        try:
```

```
624             runner, _ = seg._resolve_runner(config.indexing)
625         except NotImplementedError as e:
626             # e.g. detector_impl='native' for a family without a native runner (tracknet, or
627             # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
628             print(f"[worker] detector not available: {e}")
629             return None
630         ok, msg = runner.healthcheck()
631         if not ok:
632             print(f"[worker] detector environment not ready: {msg}")
633             return None
634         print(f"[worker] detector ready ({args.segmenter}): {msg}")
635         return runner
636
637
638     def cmd_train(args: argparse.Namespace) -> int:
639         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
640         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
641
642         Two trajectory sources (the train pipeline + harness are identical either way):
643           * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
644             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
645           * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
646             DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
647             is the M3.5 "first real training" path; only the trajectory source flips.
648         """
649         import subprocess
650
651         config = load_config(args.config) if args.config else load_config()
652         _apply_overrides(config, args.set)
653         if not _run_startup_checks(config):
654             return 2
655         storage_cfg = config.storage.model_dump()
656
657         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
658         labels_dir = os.path.join(scratch, "labels")
659         traj_dir = os.path.join(scratch, "trajectories")
660         videos_dir = os.path.join(scratch, "videos")
661         os.makedirs(labels_dir, exist_ok=True)
662         os.makedirs(traj_dir, exist_ok=True)
663
664         corpus = args.corpus_uri.rstrip("/")
665         label_uris = sorted(
666             u
667             for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
668             if u.lower().endswith(".csv")
669         )
670         if len(label_uris) < 2:
671             print(
672                 f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
673                 f"under {corpus}/labels/"
674             )
675             return 2
676
677         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
678         runner = None
679         video_by_stem: dict = {}
680         if args.trajectory_source == "detector":
```

```
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                if not vname.lower().endswith(_VIDEO_EXTS):
                    continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
                video_by_stem[os.path.splitext(vname)[0]] = vuri

        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name  # e.g. 2026-06-22_GX020094.rallies.csv
            name = _label_clip_name(fname)  # -> GX020094 (date prefix stripped)
            local_label = storage.fetch(
                uri, os.path.join(labels_dir, fname), config=storage_cfg
            )
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(
                        f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
                    )
                    continue
                local_video = storage.fetch(
                    vuri,
                    os.path.join(videos_dir, storage.parse_uri(vuri).name),
                    config=storage_cfg,
                )
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
                # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
                meta = _probe_video_fps_width(local_video)
                if meta is None:
                    print(
                        f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
                    )
                    continue
                fps, frame_width = meta
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(
                        f"[worker] detector produced no trajectory for {name!r} ? skipping."
                    )
                    continue
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": fps,
                        "frame_width": frame_width,
                    }
                )
            else:
                from backend.pipeline.detectors.stub_runner import (
                    synth_trajectory_from_labels,
                )

                traj = os.path.join(traj_dir, f"{name}_traj.csv")
```

```
                synth_trajectory_from_labels(
                    local_label, traj, fps=args.fps, frame_width=args.frame_width
                )
                specs.append(
                    {
                        "name": name,
                        "trajectory": traj,
                        "labels": local_label,
                        "fps": args.fps,
                        "frame_width": args.frame_width,
                    }
                )

        if len(specs) < 2:
            print(
                f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
            )
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = (
            "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
        )
        print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

        out_dir = os.path.join(scratch, "artifacts")
        cmd = [
            sys.executable,
            "-m",
            "training.gen0.harness",
            "--manifest",
            manifest_path,
            "--out",
            out_dir,
            "--version",
            args.version,
        ]
        if args.floor:
            floor_local = (
                storage.fetch(
                    args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
                )
                if "://" in args.floor
                else args.floor
            )
            cmd += ["--floor", floor_local]
        print("[worker] training:", " ".join(cmd))
        rc = subprocess.run(cmd).returncode
        if rc != 0:
            print(f"[worker] gen0 harness failed (rc={rc})")
            return rc

        # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
        bundle = os.path.join(out_dir, args.version)
        out_prefix = args.out_uri.rstrip("/")
        pushed = []
        for fn in sorted(os.listdir(bundle)):
            uri = f"{out_prefix}/weights/{args.version}/{fn}"
            storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
            pushed.append(uri)
        print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
        for u in pushed:
            print("   ", u)
```

```
        return 0


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(
            prog="python -m backend.worker",
            description="Storage-aware worker: pull ? run pipeline ? push.",
        )
        p.add_argument(
            "-v",
            "--verbose",
            action="store_true",
            help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
        )
        sub = p.add_subparsers(dest="cmd", required=True)

        pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
        pi.add_argument(
            "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
        )
        pi.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (outputs/<video_id>/? is appended)",
        )
        pi.add_argument(
            "--segmenter", default=None, help="default: indexing.default_segmenter"
        )
        pi.add_argument(
            "--config", default=None, help="config.json path (default: ./config.json)"
        )
        pi.add_argument(
            "--scratch", default=None, help="local scratch dir (default: a temp dir)"
        )
        pi.add_argument("--max-frames", type=int, default=None)
        pi.add_argument(
            "--done-uri",
            default=None,
            help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
            "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
            "Default-OFF: omit it for the normal local run.",
        )
        pi.add_argument(
            "--set",
            action="append",
            default=[],
            metavar="k=v",
            help="config override, e.g. --set indexing.skip_ai_handoff=true",
        )
        pi.add_argument(
            "--skip-validation",
            action="store_true",
            help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
            "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
            "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
            "`worker infer` CLI still validates.",
        )
        pi.set_defaults(func=cmd_infer)

        pc = sub.add_parser(
            "compile",
```

```
            help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged
compile-spec",
        )
        pc.add_argument(
            "--spec-uri",
            required=True,
            help="storage URI of the control-plane-staged compile-spec JSON (time-pairs +
stitching "
            "config + source uri + video_id)",
        )
        pc.add_argument(
            "--out-uri",
            default="",
            help="output ROOT (gs://? bucket); the reel is mirrored to
highlights/<video_id>/<name>. "
            "Also forwarded via --set storage.output_root; either resolves the reel path.",
        )
        pc.add_argument(
            "--done-uri",
            default=None,
            help="storage URI for the compile completion MARKER (rc + download_url), so a
remote "
            "dispatcher's bucket-poll terminates. Default-OFF: omit for a direct-CLI
stitch.",
        )
        pc.add_argument(
            "--config", default=None, help="config.json path (default: ./config.json)"
        )
        pc.add_argument(
            "--scratch", default=None, help="local scratch dir (default: a temp dir)"
        )
        pc.add_argument(
            "--set",
            action="append",
            default=[],
            metavar="k=v",
            help="config override, e.g. --set storage.output_root=gs://bucket",
        )
        pc.set_defaults(func=cmd_compile)

        pt = sub.add_parser(
            "train",
            help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
        )
        pt.add_argument(
            "--corpus-uri",
            required=True,
            help="prefix containing labels/ (+ videos/ for "
            "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
        )
        pt.add_argument(
            "--out-uri",
            required=True,
            help="output prefix (weights/<version>/ is appended)",
        )
        pt.add_argument("--version", required=True, help="semver for the artifact bundle")
        pt.add_argument(
            "--trajectory-source",
            choices=["synth", "detector"],
            default="synth",
            help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
            "detector = REAL shuttle trajectories via the resolved DetectorRunner "
            "(needs videos/ in the corpus; honours detector_impl).",
        )
        pt.add_argument(
```

```
927             "--segmenter",
928             default="wasb_hybrid",
929             help="trajectory hybrid whose detector is used for --trajectory-source
detector",
930         )
931         pt.add_argument(
932             "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
933         )
934         pt.add_argument("--config", default=None)
935         pt.add_argument("--scratch", default=None)
936         pt.add_argument(
937             "--fps",
938             type=float,
939             default=30.0,
940             help="synth trajectory fps (detector probes the video)",
941         )
942         pt.add_argument(
943             "--frame-width",
944             type=float,
945             default=1280.0,
946             help="synth trajectory frame width (detector probes the video)",
947         )
948         pt.add_argument("--set", action="append", default=[], metavar="k=v")
949         pt.set_defaults(func=cmd_train)
950         return p
951
952
953     def _setup_logging(verbose: bool) -> None:
954         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
955         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
956         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
957         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
958         entry), not at import, so importing the module never reconfigures a caller's
logging."""
959         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
960         # configured logging at import), which would leave the level at WARNING and re-
suppress the
961         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
962         logging.basicConfig(
963             level=logging.DEBUG if verbose else logging.INFO,
964             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
965             datefmt="%H:%M:%S",
966             force=True,
967         )
968
969
970     def main(argv=None) -> int:
971         args = build_parser().parse_args(argv)
972         _setup_logging(getattr(args, "verbose", False))
973         return args.func(args)
974
975
976     if __name__ == "__main__":
977         sys.exit(main())
978
```

7. assistant (2026-07-08T06:13:22.832Z)

The worker code confirms the claim's description of the structure. Now I need to verify the key

premise: whether `cmd_infer` truly writes a marker for "every pre-segmentation failure" (the claimed asymmetry), and how the compile dispatch path handles a marker-less crash. The working tree has merge conflicts, so let me read the committed merged version at HEAD.

8. user (2026-07-08T06:13:26.962Z)

```
=== git state ===
UU backend/orchestration.py
=== is HEAD the merge? ===
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
=== HEAD content OK check ===
516
```

9. assistant (2026-07-08T06:13:30.684Z)

Let me read the committed merged versions of the compute backend and orchestration compile-dispatch path.

10. user (2026-07-08T06:13:33.540Z)

```
written
  516 C:\Users\avidu\AppData\Local\Temp/cloud_run_head.py
 1460 C:\Users\avidu\AppData\Local\Temp/orchestration_head.py
 1976 total
```

11. user (2026-07-08T06:13:36.035Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10           cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11           job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12           worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13           the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14           *Fast negative path:* while the marker stays the SOLE success signal, each poll
also probes the
15           Cloud Run Executions API ? if the execution has TERMINATED without ever writing a
marker (a
16           worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
17           (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile
wait.
18        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
19           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
20
21      The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
```

```
    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.

    The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
    with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
    owner-verified on the M7 real run.
    """

    from __future__ import annotations

    import json
    import logging
    import os
    import tempfile
    import uuid
    from abc import ABC, abstractmethod
    from typing import Any, Callable, List, Optional

    from backend.compute.base import (
        CompileJobSpec,
        ComputeBackend,
        ComputeJobSpec,
        ComputeResult,
        RemoteExecutionFailed,
    )
    from backend.infrastructure.execution_policy import (
        DEFAULT_POLICY,
        ExecutionPolicy,
        PolicyTimeout,
        poll_until,
    )
    from backend.storage import fetch, get_storage, parse_uri

    LOG = logging.getLogger("compute.cloud_run")

    # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
    # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
    _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
        "deployment.compute_target=local_gpu",
        "indexing.detector_impl=native",
    )


    class CloudRunJobClient(ABC):
        """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.

        Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
        overrides the job's container ``args`` for this execution and starts it."""

        @abstractmethod
        def run_job(
            self,
            job_name: str,
            argv: List[str],
            *,
            region: str,
            project: Optional[str] = None,
        ) -> str:
            """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
```

```
return an
80              execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
81
82          @abstractmethod
83          def cancel_execution(self, execution: str) -> None:
84              """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
85              times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
86              May raise ? the caller treats every failure as best-effort (the original poll
timeout is
87              NEVER masked by a cancel error)."""
88
89          def execution_terminal_state(self, execution: str) -> Optional[str]:
90              """Return the execution's TERMINAL state as a short label (``"Failed"`` /
``"Cancelled"`` /
91              ``"Succeeded"``) if it has finished, else ``None`` (still pending/running, or
the state
92              can't be determined).
93
94              ADVISORY, not authoritative: it feeds ``run_infer``'s fast negative path so a
worker that
95              terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails
fast instead
96              of bucket-polling the full budget. The done-marker stays the ONLY success
signal.
97
98              NON-abstract with a default of ``None`` (additive / default-OFF): a client that
can't ? or
99              chooses not to ? poll the Executions API degrades transparently to today's
marker-only
100             polling. Implementations MAY raise on a transient API error ? ``run_infer``
treats a raise
101             as ``None`` (unknown ? keep waiting), so a flaky status API never fails a
healthy run."""
102             return None
103
104
105     class GoogleCloudRunJobClient(CloudRunJobClient):
106         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
107         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
108
109         def run_job(
110             self,
111             job_name: str,
112             argv: List[str],
113             *,
114             region: str,
115             project: Optional[str] = None,
116         ) -> str:
117             # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
118             # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
119             # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
120             if not project:
121                 raise RuntimeError(
122                     "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
123                     "to resolve the job path; set it on the control plane (see
get_compute_backend)."
```

```
                )
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
                raise RuntimeError(
                    "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = (
                run_v2.JobsClient()
            )  # pragma: no cover - exercised only on the real M7 run
            name = client.job_path(project, region, job_name)
            overrides = run_v2.RunJobRequest.Overrides(
                container_overrides=[
                    run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
                ]
            )
            op = client.run_job(
                request=run_v2.RunJobRequest(name=name, overrides=overrides)
            )
            return (
                getattr(op, "metadata", None)
                and getattr(op.metadata, "name", "")
                or "execution"
            )

        def cancel_execution(self, execution: str) -> None:
            # run_job falls back to the literal string "execution" when the operation
metadata carries
            # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
            # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
(where
            # google-cloud-run is absent).
            if "/executions/" not in (execution or ""):
                raise RuntimeError(
                    f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
                    "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
                    "one from the operation metadata, so there is nothing to cancel by
name)."
                )
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
                raise RuntimeError(
                    "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = (
                run_v2.ExecutionsClient()
            )  # pragma: no cover - exercised only on a real run
            client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))

        def execution_terminal_state(self, execution: str) -> Optional[str]:
            # Advisory status probe (see the ABC docstring): fetch the execution and
classify it via the
            # pure, unit-tested ``_classify_execution_state``. A non-resolvable name /
absent SDK returns
            # None (can't probe ? keep marker-polling) ? unlike run_job/cancel this never
raises on bad
            # input, because the probe is best-effort and must not fail a healthy run.
```

```
            if "/executions/" not in (execution or ""):
                return None
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception:  # pragma: no cover - real SDK not present in CI
                return None
            client = (
                run_v2.ExecutionsClient()
            )  # pragma: no cover - exercised only on a real run
            exe = client.get_execution(  # pragma: no cover - real run only
                request=run_v2.GetExecutionRequest(name=execution)
            )
            return _classify_execution_state(exe)  # pragma: no cover - real run only


    def _classify_execution_state(exe: Any) -> Optional[str]:
        """Map a Cloud Run ``run_v2.Execution`` to a terminal state label (``"Failed"`` /
``"Cancelled"``
        / ``"Succeeded"``), or ``None`` if it is still pending/running. PURE (no
SDK/network) so it is
        unit-tested against both real proto-plus ``Execution`` objects and lightweight fakes
? the actual
        ``get_execution`` fetch is the only part that can't run in CI.

        ``google-cloud-run`` is a **proto-plus** library, so a ``google.protobuf.Timestamp``
field is
        marshalled to a native Python ``datetime`` when set and to ``None`` when unset ? NOT
to a proto
        ``Timestamp`` with a ``.seconds`` field (a ``datetime`` has ``.second``, singular).
Terminality is
        therefore "``completion_time`` is not None"; the per-task counts (plain ints) then
classify it,
        with a cancel taking precedence (a cancelled execution reports ``cancelled_count >
0``)."""
        if getattr(exe, "completion_time", None) is None:
            return None  # no terminal completion timestamp yet ? still pending/running
        if getattr(exe, "cancelled_count", 0):
            return "Cancelled"
        if getattr(exe, "failed_count", 0):
            return "Failed"
        if getattr(exe, "succeeded_count", 0):
            return "Succeeded"
        return "Failed"  # terminal but no positive success ? treat as failed (conservative)


    class CloudRunCompute(ComputeBackend):
        """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""

        def __init__(
            self,
            *,
            run_client: CloudRunJobClient,
            job_name: str,
            region: str,
            project: Optional[str] = None,
            storage_config: Optional[dict] = None,
            policy: ExecutionPolicy = DEFAULT_POLICY,
            token: Optional[str] = None,
            container_overrides: Optional[tuple[str, ...]] = None,
        ) -> None:
            self._client = run_client
            self._job_name = job_name
            self._region = region
            self._project = project
            self._storage_config = storage_config or {}
```

```
            self._policy = policy
            # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
            # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
            self._token = token
            self._container_overrides = (
                _DEFAULT_CONTAINER_OVERRIDES
                if container_overrides is None
                else tuple(container_overrides)
            )

        def run_infer(
            self,
            spec: ComputeJobSpec,
            *,
            on_wait: "Callable[[float], None] | None" = None,
        ) -> ComputeResult:
            out_root = spec.out_uri.rstrip("/")
            token = self._token or uuid.uuid4().hex
            done_uri = f"{out_root}/_dispatch/{token}.done"
            argv: List[str] = [
                "infer",
                "--video-uri",
                spec.video_uri,
                "--out-uri",
                spec.out_uri,
                "--done-uri",
                done_uri,
            ]
            if spec.segmenter:
                argv += ["--segmenter", spec.segmenter]
            if spec.skip_validation:
                # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
                # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
                argv.append("--skip-validation")
            for kv in (*spec.config_overrides, *self._container_overrides):
                argv += ["--set", kv]

            execution = self._client.run_job(
                self._job_name, argv, region=self._region, project=self._project
            )
            LOG.info(
                "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
                self._job_name,
                execution,
                done_uri,
            )

            marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-
infer")
            video_id = str(marker.get("video_id", ""))
            # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
            # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
            rc = int(marker.get("returncode", 1))
            if not video_id:
                LOG.warning(
                    "cloud-run: job=%s completion marker present but empty/unreadable (no "
                    "video_id) ? treating as failure",
                    self._job_name,
                )
```

```
                rc = rc or 1
            outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
            report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
            LOG.info(
                "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
            )
            return ComputeResult(
                returncode=rc,
                outputs_uri=outputs_uri,
                video_id=video_id,
                report=report,
                execution=str(execution or ""),
            )

        def run_compile(
            self,
            spec: "CompileJobSpec",
            *,
            on_wait: "Callable[[float], None] | None" = None,
        ) -> ComputeResult:
            """Dispatch a reel-COMPILE (#521): run ``python -m backend.worker compile`` on
the L4 (NVENC),
            bucket-poll the done-marker, and return the worker's rc + marker. Reuses the
EXACT
            dispatch?poll?marker machinery as ``run_infer`` (``_await_marker`` ? crash-fast
+ timeout
            rescue), so a stitch that crashes or overruns fails the same way a detection
does ? not a
            ~65-min opaque hang."""
            out_root = spec.out_uri.rstrip("/")
            token = self._token or uuid.uuid4().hex
            # Distinct marker prefix from infer's _dispatch/ so a compile + a detection for
the same
            # bucket can never collide on a marker.
            done_uri = f"{out_root}/_compile/{token}.done"
            argv: List[str] = [
                "compile",
                "--out-uri",
                spec.out_uri,
                "--spec-uri",
                spec.spec_uri,
                "--done-uri",
                done_uri,
            ]
            # Compile carries NO container-inline overrides: cmd_compile never consults
compute_target and
            # never re-dispatches, so the only overrides are the CP-forwarded
storage/encoder knobs.
            for kv in spec.config_overrides:
                argv += ["--set", kv]

            execution = self._client.run_job(
                self._job_name, argv, region=self._region, project=self._project
            )
            LOG.info(
                "cloud-run: dispatched COMPILE job=%s exec=%s; bucket-polling %s",
                self._job_name,
                execution,
                done_uri,
            )
            marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-
compile")
            video_id = str(marker.get("video_id", "") or spec.video_id)
            rc = int(marker.get("returncode", 1))
            LOG.info(
```

```
350                 "cloud-run: COMPILE job=%s done (video_id=%s rc=%d)",
351                 self._job_name,
352                 video_id,
353                 rc,
354             )
355             # The marker carries the worker's stitch result (download_url / reel_uri /
status); surface it
356             # as ``report`` so the dispatcher returns the same {status, filepath,
download_url} shape the
357             # SPA expects. ``outputs_uri`` is the mirrored reel URI when present.
358             return ComputeResult(
359                 returncode=rc,
360                 outputs_uri=str(marker.get("reel_uri", "") or ""),
361                 video_id=video_id,
362                 report=marker,
363                 execution=str(execution or ""),
364             )
365
366         def _await_marker(
367             self,
368             done_uri: str,
369             execution: object,
370             on_wait: "Callable[[float], None] | None",
371             *,
372             label: str,
373         ) -> dict:
374             """Bucket-poll ``done_uri`` to a completion marker ? shared by ``run_infer`` and
375             ``run_compile``. ``on_wait`` is the #482 live heartbeat (each still-waiting tick
reaches the
376             caller so job.progress moves). A ``RemoteExecutionFailed`` (the fast negative
path in
377             ``_poll_check``) is NOT a ``PolicyTimeout`` so it propagates uncaught ? the
dispatch handler
378             maps it to a distinct failure; a genuine poll TIMEOUT is handled (rescue-or-
cancel)."""
379             try:
380                 return poll_until(
381                     lambda: self._poll_check(done_uri, str(execution or "")),
382                     self._policy,
383                     label=label,
384                     logger=LOG,
385                     on_tick=on_wait,
386                 )
387             except PolicyTimeout as timeout_exc:
388                 return self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
389
390         def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
391             """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
392
393             The done-marker is the SOLE success signal and is checked FIRST ? a present
marker always
394             wins (even if the execution's status later flips), so a worker that wrote a
SUCCESS *or a
395             FAILURE* marker resolves through the normal path with its real returncode. Only
when the
396             marker is absent do we consult the execution status: a worker that TERMINATED
without ever
397             writing a marker (crash / OOM / an old image argparse-aborting on an unknown
flag) would
398             otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
before failing
399             opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
400             failure.
```

```
401
402             Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
403             ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,
no marker
404             will ever appear)."""
405             marker = self._read_marker(done_uri)
406             if marker is not None:
407                 return marker  # success signal ? authoritative even if status later flips
408             state = self._terminal_execution_state(execution)
409             if state is None:
410                 return None  # still pending/running, or status unknown ? keep bucket-
polling
411             # The execution is terminal but no marker is present. Re-check the marker ONCE
to close the
412             # race where the worker wrote it between our two reads (mirrors the post-timeout
rescue in
413             # _handle_poll_timeout). If it is genuinely absent, the process is gone and no
marker will
414             # EVER appear ? fail fast + diagnostically rather than wait out the whole poll
budget.
415             marker = self._read_marker(done_uri)
416             if marker is not None:
417                 return marker
418             raise RemoteExecutionFailed(
419                 f"cloud-run: job={self._job_name} execution={execution or '<unknown>'}
reached "
420                 f"terminal state {state!r} with NO completion marker ? the worker exited
before "
421                 "writing one (crash / OOM / pre-marker abort, or a failed marker upload on
an "
422                 "otherwise-Succeeded run). The execution is over, so the marker will never
appear; "
423                 "inspect the Cloud Run Job execution log to diagnose: "
424                 f"`gcloud run jobs executions describe {execution or '<execution>'} "
425                 f"--region {self._region}`."
426             )
427
428         def _terminal_execution_state(self, execution: str) -> Optional[str]:
429             """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
raising or
430             flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
431             so any exception ? and an empty/unresolvable execution name ? degrades to
``None`` (unknown
432             ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
433             if not execution:
434                 return None
435             try:
436                 return self._client.execution_terminal_state(execution)
437             except Exception:  # noqa: BLE001 ? advisory probe: never let it kill the
bucket-poll
438                 LOG.debug(
439                     "cloud-run: execution status probe failed for %s ? treating as unknown",
440                     execution,
441                     exc_info=True,
442                 )
443                 return None
444
445         def _handle_poll_timeout(
446             self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
447         ) -> dict:
448             """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
449
450             The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
```

```
451             ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
452             ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
453             is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
454             L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
455             execution name so the operator can verify the state in the GCP console. A cancel
failure
456             NEVER masks the original timeout."""
457             try:
458                 late = self._read_marker(done_uri)
459             except Exception:  # noqa: BLE001 ? the final check is best-effort too: absence
? failure
460                 LOG.warning(
461                     "cloud-run: final post-timeout marker check failed ? treating as
absent",
462                     exc_info=True,
463                 )
464                 late = None
465             if late is not None:
466                 LOG.warning(
467                     "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
468                     "? rescuing the completed result",
469                     self._job_name,
470                     execution,
471                 )
472                 return late
473             try:
474                 self._client.cancel_execution(execution)
475             except Exception as cancel_exc:  # noqa: BLE001 ? best-effort: never mask the
timeout
476                 cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
477                 LOG.warning(
478                     "cloud-run: best-effort cancel of execution=%s failed: %s",
479                     execution,
480                     cancel_exc,
481                     exc_info=True,
482                 )
483             else:
484                 cancel_note = "cancel requested"
485                 LOG.info("cloud-run: requested cancel of execution=%s", execution)
486             raise PolicyTimeout(
487                 f"{timeout_exc} [cloud-run job={self._job_name} "
488                 f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
489                 "state in the GCP console]"
490             ) from timeout_exc
491
492         # --- storage-backed completion signal (bucket-poll) ---
493         def _read_marker(self, done_uri: str) -> Optional[dict]:
494             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
495             ref = parse_uri(done_uri)
496             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
497                 return None
498             return (
499                 self._read_json(done_uri) or {}
500             )  # empty dict still terminates the poll (present == done)
501
502         def _read_json(self, uri: str) -> dict:
503             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
```

```
504             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
505             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
506             try:
507                 ref = parse_uri(uri)
508                 with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
509                     dst = os.path.join(td, ref.name or "obj.json")
510                     local = fetch(uri, dst, config=self._storage_config)
511                     with open(local, "r", encoding="utf-8") as f:
512                         data: Any = json.load(f)
513                 return data if isinstance(data, dict) else {}
514             except Exception:
515                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
516                 return {}
517
```

12. assistant (2026-07-08T06:14:00.927Z)

The `run_compile` path reuses the exact `_await_marker`/`_poll_check` crash-fast machinery as `run_infer`. This is central to refuting the claim's asymmetry premise. Let me verify how `cmd_infer`'s own early failures behave, and how `_stitch_reel` and the spec-staging work in orchestration.

13. user (2026-07-08T06:14:07.457Z)

```
1190:def _stitch_reel(
1279:def _dispatch_compile_remote(
1295:    from backend.compute import CompileJobSpec, get_compute_backend
1319:        return _stitch_reel(
1341:    spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
1348:            storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
1351:            "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
exc_info=True
1365:    spec = CompileJobSpec(
1367:        spec_uri=spec_uri,
1381:        result = compute.run_compile(
1458:    return _stitch_reel(
```

14. user (2026-07-08T06:14:09.851Z)

```
1190    def _stitch_reel(
1191        cfg,
1192        stitching,
1193        video_id: str,
1194        source_ref: str,
1195        time_pairs: List[Tuple[float, float]],
1196        out_name: str,
1197        locale: Optional[str],
1198        notify,
1199    ) -> Dict[str, Any]:
1200        """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
1201        (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
1202        needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
1203        with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
1204        trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
1205        returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
1206        ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
```

```
failure)."""
1207        import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
1208
1209        output_dir = (
1210            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1211        )
1212        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1213        # unknown/absent locale keeps the configured default unchanged.
1214        localized_watermark = localize_watermark(stitching.watermark, locale)
1215        compiler = _main.VideoCompiler(
1216            output_dir,
1217            begin_padding=stitching.begin_padding,
1218            end_padding=stitching.end_padding,
1219            watermark=localized_watermark,
1220            encoder=getattr(stitching, "encoder", "libx264"),
1221        )
1222
1223        notify("stitching")
1224        local_src = ""
1225        try:
1226            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1227            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1228            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1229            local_src = storage.acquire_local_input(
1230                source_ref, getattr(cfg, "storage", None)
1231            )
1232            # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1233            # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1234            with _LOCAL_COMPUTE_LANE:
1235                out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1236        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1237            logger.error(
1238                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1239            )
1240            return {
1241                "status": "failed",
1242                "message": f"Highlight stitching failed: {e}",
1243                "video_id": video_id,
1244            }
1245        finally:
1246            if local_src:
1247                storage.cleanup_acquired_local_input(
1248                    source_ref, local_src, getattr(cfg, "storage", None)
1249                )
1250
1251        # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1252        # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1253        # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1254        # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
1255        # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
1256        reel_uri = reel_object_uri(cfg, video_id, out_name)
1257        if reel_uri:
```

```
            try:
                notify("uploading reel")
                # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
                # project/client) is used, not a fresh default.
                storage.put(out_path, reel_uri, config=_storage_settings(cfg))
            except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
                logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)

        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
        notify("finished")
        return {
            "status": "success",
            "filepath": out_path,
            "download_url": download_url,
            "video_id": video_id,
            "reel_uri": reel_uri or "",
        }


    def _dispatch_compile_remote(
        cfg,
        db,
        video_id: str,
        segment_ids: List[int],
        output_filename: str,
        locale: Optional[str],
        notify,
    ) -> Dict[str, Any]:
        """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-
plane. The
        control-plane owns the DB, so it RESOLVES the reel's segments here
(``_resolve_compile``), stages a
        small compile-spec (resolved time-pairs + the resolved stitching/watermark config +
the source URI)
        to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-
polls the
        done-marker, and returns the same ``{status, filepath, download_url}`` shape the SPA
expects. The
        worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
        ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
        from backend.compute import CompileJobSpec, get_compute_backend

        notify("resolving")
        try:
            video, kept, out_name = _resolve_compile(
                db, cfg, video_id, segment_ids, output_filename
            )
        except _CompileError as e:
            return {"status": "failed", "message": e.detail, "video_id": video_id}

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
        storage_cfg = getattr(cfg, "storage", None)
        out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
        compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else
None
        if compute is None:
            # Compile was placed on the L4 but the cloud dispatch isn't actually wired here
(no output
            # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail ?
a slow reel
```

```
            # beats no reel. (The placement guard means this is only reachable in a mis-
provisioned setup.)
            logger.warning(
                "[compile] compile placed on the L4 but no cloud backend available
(out_root=%r) ? "
                "stitching in-process on the control-plane instead.",
                out_root,
            )
            time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
            return _stitch_reel(
                cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
            )

        # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe
on the wire);
        # the stitching config is the control-plane's RESOLVED one, with the encoder swapped
to the L4's
        # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-encodes
the reel.
        stitch_dump = stitching.model_dump()
        stitch_dump["encoder"] = getattr(
            getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
        )
        spec_payload = {
            "video_id": video_id,
            "video_uri": video["filepath"],
            "output_name": out_name,
            "locale": locale,
            "time_pairs": [
                [float(s["start_time"]), float(s["end_time"])] for s in kept
            ],
            "stitching": stitch_dump,
        }
        token = uuid.uuid4().hex
        spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
        try:
            notify("staging compile spec")
            with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
                local_spec = os.path.join(td, "spec.json")
                with open(local_spec, "w", encoding="utf-8") as f:
                    json.dump(spec_payload, f)
                storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
        except Exception as e:  # noqa: BLE001 ? a staging failure is a clean job failure,
not a crash
            logger.error(
                "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
exc_info=True
            )
            return {
                "status": "failed",
                "message": f"cloud compile: could not stage the compile spec: {e}",
                "video_id": video_id,
            }

        # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the
spec, mirror the
        # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB
knobs for detect.
        overrides = [f"storage.output_root={out_root}"]
        backend_name = (getattr(storage_cfg, "backend", "") or "").strip()
        if backend_name:
            overrides.append(f"storage.backend={backend_name}")
        spec = CompileJobSpec(
            out_uri=out_root,
```

```
            spec_uri=spec_uri,
            video_id=video_id,
            output_name=out_name,
            config_overrides=tuple(overrides),
        )

        notify("dispatching stitch (cloud_run)")
        logger.info(
            "[compile] dispatching stitch for video_id=%s -> Cloud Run Job (out=%s, %d
clip(s))",
            video_id,
            out_root,
            len(kept),
        )
        try:
            result = compute.run_compile(
                spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
            )
        except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
            logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
            return {
                "status": "failed",
                "message": f"cloud compile dispatch error: {e}",
                "video_id": video_id,
            }
        if result.returncode != 0:
            msg = str(
                result.report.get("message")
                or f"cloud compile job failed (returncode {result.returncode})."
            )
            return {"status": "failed", "message": msg, "video_id": video_id}

        download_url = str(
            result.report.get("download_url")
            or f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
        )
        logger.info(
            "[COMPILE] cloud_run stitched: execution=%s out=%s video_id=%s",
            result.execution,
            result.outputs_uri,
            video_id,
        )
        notify("finished")
        return {
            "status": "success",
            "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
            "download_url": download_url,
            "video_id": video_id,
            "compute": {
                "path": "cloud_run",
                "execution": result.execution,
                "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
                "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
            },
        }


    def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
        """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel, and return
        the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
        result for an EXPECTED problem (missing video / no segments / a stitch error) so the
SPA always gets
```

```
1427        a clean verdict instead of a worker-crash error.
1428
1429        #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``)
and cloud
1430        dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
1431        (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip
reel on the
1432        2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process
here, unchanged."""
1433        notify = progress or (lambda stage: None)
1434        params = json.loads(job.get("params") or "{}")
1435        video_id = job.get("video_id") or ""
1436        segment_ids = params.get("segment_ids") or []
1437        locale = params.get("locale")
1438        output_filename = params.get("output_filename", "")
1439
1440        if (
1441            resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
1442            and _cloud_available(cfg)
1443        ):
1444            return _dispatch_compile_remote(
1445                cfg, db, video_id, segment_ids, output_filename, locale, notify
1446            )
1447
1448        notify("resolving")
1449        try:
1450            video, kept, out_name = _resolve_compile(
1451                db, cfg, video_id, segment_ids, output_filename
1452            )
1453        except _CompileError as e:
1454            return {"status": "failed", "message": e.detail, "video_id": video_id}
1455
1456        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1457        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1458        return _stitch_reel(
1459            cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
```

15. assistant (2026-07-08T06:15:06.323Z)

I now have everything needed to evaluate the claim. Let me consolidate the analysis.

The claim asserts a defect at `backend/worker/__main__.py:488`: `cmd_compile` writes its done-marker only after `_stitch_reel` returns, so a raise BEFORE that (spec fetch/parse, `time_pairs` coercion, `StitchingConfig` init) exits the worker with no marker ? claimed to be an **asymmetry** vs `cmd_infer`, whose `_fail()` allegedly "writes a real-rc marker for every pre-segmentation failure."

The code structure the claim describes is accurate, but its load-bearing premise ? the asymmetry ? is false:

1. **`cmd_infer` does NOT write a marker for "every pre-segmentation failure."** `_fail()` (worker.py:301) is only ever invoked from inside the try block that starts at line 332 ? i.e. only for validation failure (`_fail(2)`, line 338), unknown segmenter (line 346), and a segmenter `Failure` (line 354). Everything before that raises or returns with **no marker**:
   - `_run_startup_checks` returns 2 at line 248 (no marker),
   - `storage.fetch(...)` ? the PULL at lines 265-267 ? is the **direct analog** of `_read_compile_spec`'s fetch; a truncated/corrupt/missing video raises here with no marker,
   - `compute_video_id` (271), `validator_registry.run_all_with_context` (289), `Database(...)`/`add_video` (293-294) all raise with no marker.
   So `cmd_infer` and `cmd_compile` are **symmetric**, not asymmetric ? both exit `main()` (line 973, unwrapped) with a traceback and no marker on an early fetch/coercion failure.

2. **The compile path reuses the SAME crash-fast probe as infer.** `run_compile` (cloud_run.py:307-364) calls the identical `_await_marker` (line 346) that `run_infer` uses

(line 282). `_await_marker` ? `poll_until(_poll_check)` ? `_poll_check` (390-426) probes the Executions API and raises `RemoteExecutionFailed` the moment the execution is terminal with no marker. A marker-less compile crash is caught by the exact #515 fast-negative path ? the invariant "mirror the existing detection dispatch" is literally satisfied.

3. **The orchestration layer converts the resulting exception into a clean failed job.** `_dispatch_compile_remote` wraps `compute.run_compile(...)` in try/except (orchestration.py:1380-1390) that maps ANY exception (`RemoteExecutionFailed`, `PolicyTimeout`, etc.) to `{status:"failed", ...}` ? never a crash/500.

4. **The actual expected failure mode (a stitch/ffmpeg error) is already clean.** `_stitch_reel` catches it internally and returns `status="failed"` (orchestration.py:1236-1244, "never raises for an expected ffmpeg failure"); `cmd_compile` then sets rc=3 and writes the marker (worker.py:513-520). The "stitch failure must be a clean failed-job" invariant holds.

5. **The claim's fallback ("if the probe is unavailable/delayed, the poll hangs ~65 min") describes the pre-existing, documented, accepted default-OFF degradation** ? the Executions-API probe is explicitly advisory/best-effort ("degrades transparently to today's marker-only polling", cloud_run.py:98-102, 428-443). It is identical for infer's marker-less PULL crash and is not a new defect introduced by #521.

6. **The spec producer is trusted:** `_dispatch_compile_remote` builds `time_pairs` as `[[float(start), float(end)], ...]` (orchestration.py:1335-1337), so malformed pairs from the control-plane are not reachable; only a transient truncated read could trigger the path ? the same class as infer's truncated video PULL.

The claim identifies a real code structure but mischaracterizes it as a defect via a false asymmetry premise. The behavior is symmetric with the infer dispatch it mirrors, is backstopped by the shared crash-fast probe and the orchestration try/except, and preserves every stated invariant.

16. user (2026-07-08T06:15:16.589Z)

Structured output provided successfully